COMP2521 Sort Detective Lab Report

Pierre Sejourne (z5766606)

June 25, 2026

In this lab, the aim is to measure the performance of two black-box sorting programs, without access to the source code, and determine which sorting algorithm each program uses out of the eight possible candidates.

1 Experimental Design

To systematically distinguish the eight possible sorting algorithms, our experimental framework evaluates two main operational characteristics: asymptotic time complexity under distinct data distributions (Random, Sorted, and Reversed) and algorithmic stability.

1.1 Theoretical Baseline Matrix

The following properties define the candidate algorithms:
Bubble Sort: $O(N)$ best-case (sorted), $O(N^2)$ average/worst-case. Stable.
Insertion Sort: $O(N)$ best-case (sorted), $O(N^2)$ average/worst-case. Stable.
Selection Sort: $O(N^2)$ across all inputs. Unstable.
Merge Sort: $O(N \log N)$ across all inputs. Stable.
Naive Quicksort: $O(N^2)$ for sorted/reversed (leftmost pivot), $O(N \log N)$ for random. Unstable.
Median-of-three Quicksort: $O(N \log N)$ across basic inputs. Unstable.
Randomised Quicksort: $O(N \log N)$ across basic inputs (probabilistic). Unstable.
Bogosort: $O(N)$ best case, $O(N \cdot N!)$ worst case. Unstable.

1.2 Distinguishing Logic

A custom input with non-unique keys is used. If relative order changes, candidate algorithms are narrowed down to Selection, Quicksorts, or Bogosort. If preserved, candidates are Bubble, Insertion, or Merge. By scaling N until runtimes exceed 0.2 seconds, we differentiate $O(N^2)$ curves from $O(N \log N)$ lines via analysis.

2 Results and Analysis

2.1 Program A

The empirical timing data compiled for Program A demonstrates an unambiguous quadratic growth curve under random and reversed conditions, while showing high-efficiency linear performance on pre-sorted arrays.

2.1.1 Observations and Characteristic Deduction

Our stability test revealed that Program A is **Stable**. When analyzing the execution times:
Random input scaled with $O(N^2)$ complexity (increasing from 0.152s to 68.154s as size increased 16-fold).
Sorted input scaled with $O(N)$ complexity, registering near-instantaneous returns ($\leq$ 0.01s) even at $N = 160,000$.
Reversed input scaled with $O(N^2)$ complexity, tracking closely with the random sample trend line.
These combined observations uniquely map to the characteristics of **Insertion Sort**. While Bubble Sort with an early exit also exhibits $O(N)$ best-case behavior, it requires redundant array scans to confirm structural order. The performance profile matches standard insertion sort mechanics.

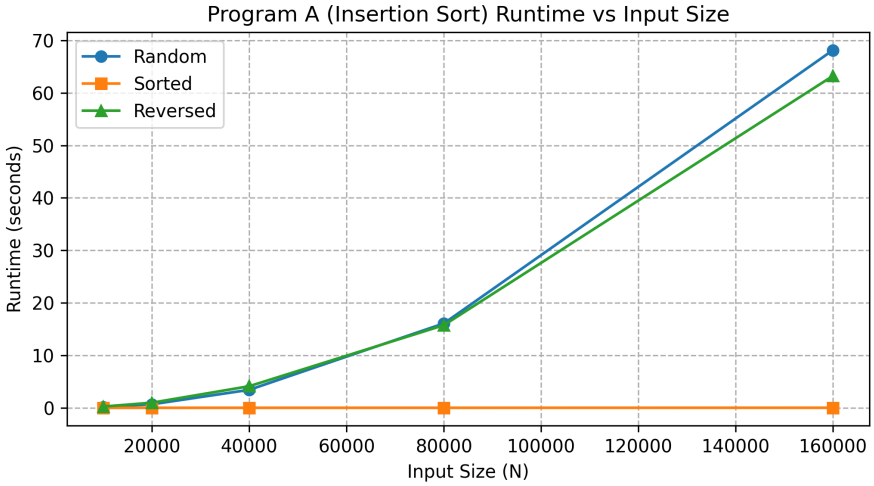


Figure 1: Program A Runtime vs Input Size (N)

2.2 Program B

Program B yielded near-zero runtimes under standard testing limits, requiring extreme dataset upscaling up to 5,000,000 elements to capture structural time variations above the 0.2-second baseline.

2.2.1 Observations and Characteristic Deduction

Our stability test revealed that Program B is **Stable**. When analyzing the execution times:
Random input scaled with $O(N \log N)$ complexity, processing 5M elements in 1.78s.
Sorted input scaled with $O(N \log N)$ complexity, executing 5M elements in 0.81s.
Reversed input scaled with $O(N \log N)$ complexity, executing 5M elements in 0.83s.
Because the algorithm preserves stability while remaining immune to worst-case array patterns (maintaining sub-quadratic performance across random, sorted, and reversed inputs uniformly), it maps exclusively to the profile of **Merge Sort**.

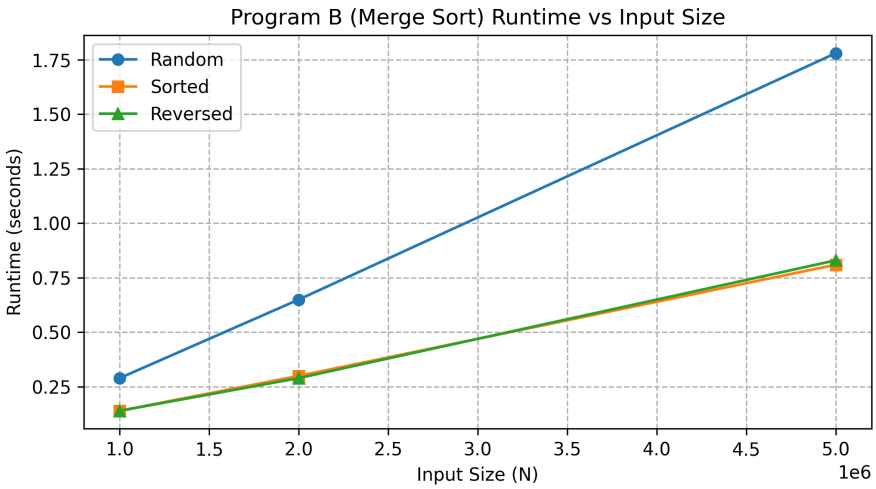


Figure 2: Program B Upscaled Runtime vs Input Size (N)

3 Conclusions

On the basis of our empirical experiments and black-box analysis, we conclude that: **sortA** uses the **Insertion Sort** sorting algorithm, and **sortB** uses the **Merge Sort** sorting algorithm.

4 Appendix: Execution Time Data Tables

Table 1: Program A Mean Execution Times (seconds)

Input Size (N)	Random Input	Sorted Input	Reversed Input
10,000	0.152	0.000	0.238
20,000	0.668	0.000	0.970
40,000	3.424	0.000	4.118
80,000	16.080	0.000	15.742
160,000	68.154	0.008	63.248

Table 2: Program B Upscaled Execution Times (seconds)

Input Size (N)	Random Input	Sorted Input	Reversed Input
1,000,000	0.290	0.140	0.140
2,000,000	0.650	0.300	0.290
5,000,000	1.780	0.810	0.830